Deep Learning Laboratory

External Examination - Questions, Algorithms and Programs

Q	Program file
1	Answer01_FFNN_Classification.py
2	Answer02_FFNN_MNIST_Accuracy_Plot.py
3	Answer03_FFNN_Activation_Comparison.py
4	Answer04_FFNN_Xavier_Kaiming_LossCurves.py
5	Answer05_CNN_MNIST_Accuracy.py
6	Answer06_Xavier_Kaiming_Training_Comparison.py
7	Answer07_CNN_Architecture_MNIST.py
8	Answer08_CNN_Correct_Incorrect_Visualization.py
9	Answer09_RNN_vs_LSTM_IMDB.py
10	Answer10_RNN_vs_GRU.py

Question 1

Implement a Feed Forward Neural Network for classification using an appropriate dataset. Train the network and evaluate its performance using accuracy.

Algorithm

Step 1: Import TensorFlow/Keras and fix the random seed for repeatable results.

Step 2: Load the CIFAR-10 dataset (50,000 training and 10,000 test colour images of 32x32 pixels, 10 classes).

Step 3: Normalize pixel values to the range 0-1 by dividing by 255.

Step 4: One-hot encode the class labels with to_categorical.

Step 5: Build the FFNN: Input(32x32x3) -> Flatten -> Dense(512, ReLU) -> Dense(10, Softmax).

Step 6: Compile with the Adam optimizer and categorical cross-entropy loss, tracking accuracy.

Step 7: Train the model on the training data with a validation split.

Step 8: Evaluate on the test set and print the test accuracy.

Program (Answer01_FFNN_Classification.py)

```
# Q1: Feed Forward Neural Network for classification using CIFAR-10
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense
from tensorflow.keras.utils import to_categorical

tf.keras.utils.set_random_seed(42)

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    Input(shape=(32, 32, 3)),
    Flatten(),
    Dense(512, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])

model.fit(x_train, y_train, epochs=15, batch_size=128,
          validation_split=0.1, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")
print("Experiment completed successfully!")
```

Question 2

Implement a Feed Forward Neural Network for handwritten digit classification using dataset. Display the test accuracy and plot the training and validation accuracy.

Algorithm

Step 1: Import libraries and set the random seed.

Step 2: Load the MNIST dataset (60,000 training and 10,000 test images of 28x28 grayscale digits).

Step 3: Normalize pixel values to 0-1.

Step 4: Build the FFNN: Input(28x28) -> Flatten -> Dense(128, ReLU) -> Dense(10, Softmax).

Step 5: Compile with Adam and sparse categorical cross-entropy, metric = accuracy.

Step 6: Train with a validation split so that validation accuracy is recorded in the history.

Step 7: Evaluate on the test set and display the test accuracy.

Step 8: Plot training accuracy and validation accuracy against epochs.

Program (Answer02_FFNN_MNIST_Accuracy_Plot.py)

```
# Q2: FFNN for handwritten digit classification using MNIST
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(x_train, y_train, epochs=8, batch_size=128,
                    validation_split=0.1, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q2_accuracy.png")
plt.show()
print("Experiment completed successfully!")
```

Question 3

Implement a Feed Forward Neural Network with at least one hidden layer. Experiment with different activation functions and compare the classification performance.

Algorithm

Step 1: Load MNIST, normalize to 0-1 and use a fixed 20,000-sample training subset for speed.

Step 2: Define the list of activation functions to test: ReLU, Sigmoid and Tanh.

Step 3: For each activation: reset the random seed so every run is comparable.

Step 4: Build Input -> Flatten -> Dense(128, activation) -> Dense(10, Softmax) (one hidden layer).

Step 5: Compile with Adam and sparse categorical cross-entropy; train with identical settings.

Step 6: Evaluate each model on the test set and store accuracy and loss.

Step 7: Print a comparison table and plot validation accuracy for each activation.

Step 8: Conclude which activation gives the best classification performance.

Program (Answer03_FFNN_Activation_Comparison.py)

```
# Q3: Compare ReLU, Sigmoid and Tanh activation functions in an FFNN
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Use the same subset and training settings for each activation.
x_train = x_train[:20000]
y_train = y_train[:20000]

activations = ["relu", "sigmoid", "tanh"]
results = []

for activation in activations:
    print(f"\nTraining with {activation} activation")
    tf.keras.utils.set_random_seed(42)

    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation=activation),
        Dense(10, activation="softmax")
    ])

    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    history = model.fit(
        x_train, y_train, epochs=5, batch_size=128,
        validation_split=0.1, verbose=1
    )

    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results.append((activation, test_accuracy, test_loss))
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")
    print(f"Test loss: {test_loss:.4f}")

names = [item[0] for item in results]
accuracies = [item[1] * 100 for item in results]

print("\nActivation\tTest accuracy\tTest loss")
```

```
for name, accuracy, loss in results:
    print(f"{name}\t\t{accuracy:.4f}\t\t{loss:.4f}")

plt.figure(figsize=(8, 5))
plt.bar(names, accuracies)
plt.title("Test Accuracy for Different Activation Functions")
plt.xlabel("Activation function")
plt.ylabel("Test accuracy (%)")
plt.ylim(0, 100)
plt.grid(axis="y")
plt.tight_layout()
plt.savefig("Q3_activation_comparison.png")
plt.show()
print("Experiment completed successfully!")
```

Question 4

Implement a Feed Forward Neural Network using Xavier initialization and Kaiming initialization separately. Plot the loss curves and determine which initialization provides better convergence.

Algorithm

Step 1: Load MNIST, normalize and use a 20,000-sample training subset.

Step 2: Define two initializers: Xavier (glorot_uniform) and Kaiming (he_normal).

Step 3: For each initializer: reset the seed and build the same FFNN with kernel_initializer set on the hidden layers.

Step 4: Compile with Adam and sparse categorical cross-entropy; train with validation split.

Step 5: Record the training/validation loss history and the test accuracy of each model.

Step 6: Plot the loss curves of both initializations on the same graph.

Step 7: Compare final loss and accuracy; the initializer with lower loss / faster drop converges better.

Program (Answer04_FFNN_Xavier_Kaiming_LossCurves.py)

```
# Q4: Compare Xavier and Kaiming initialization in an FFNN
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Smaller subset keeps execution time manageable.
x_train = x_train[:20000]
y_train = y_train[:20000]

initializers = {
    "Xavier": "glorot_uniform",
    "Kaiming": "he_normal"
}
histories = {}
results = {}

for name, initializer in initializers.items():
    print(f"\nTraining with {name} initialization")
    tf.keras.utils.set_random_seed(42)

    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation="relu", kernel_initializer=initializer),
        Dense(10, activation="softmax")
    ])

    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    history = model.fit(
        x_train, y_train, epochs=8, batch_size=128,
        validation_split=0.1, verbose=1
    )
    histories[name] = history.history
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results[name] = (test_loss, test_accuracy)
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")
    print(f"Test loss: {test_loss:.4f}")

plt.figure(figsize=(8, 5))
```

```
for name in initializers:
    plt.plot(histories[name]["loss"], label=f"{name} training")
plt.title("Training Loss: Xavier vs Kaiming")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q4_training_loss.png")
plt.show()

plt.figure(figsize=(8, 5))
for name in initializers:
    plt.plot(histories[name]["val_loss"], label=f"{name} validation")
plt.title("Validation Loss: Xavier vs Kaiming")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q4_validation_loss.png")
plt.show()

print("\nMethod\t\tTest accuracy\tTest loss")
for name, (loss, accuracy) in results.items():
    print(f"{name}\t\t{accuracy * 100:.2f}%\t\t{loss:.4f}")

best = min(histories, key=lambda key: min(histories[key]["val_loss"]))
print("Lowest validation loss during training:", best)
print("Compare the curves; do not assume one initializer always wins.")
print("Experiment completed successfully!")
```

Question 5

Implement a Convolutional Neural Network (CNN) for handwritten digit classification using the MNIST dataset. Evaluate the model using test accuracy.

Algorithm

Step 1: Load MNIST, normalize to 0-1 and reshape to (28, 28, 1).

Step 2: One-hot encode the labels.

Step 3: Build a LeNet-like CNN: Conv2D(6, 5x5, ReLU) -> MaxPool(2x2) -> Conv2D(16, 5x5, ReLU) -> MaxPool(2x2) -> Flatten -> Dense layers -> Dense(10, Softmax).

Step 4: Compile with Adam and categorical cross-entropy.

Step 5: Train the CNN for several epochs with a validation split.

Step 6: Evaluate on the test set and print the test accuracy.

Program (Answer05_CNN_MNIST_Accuracy.py)

```
# Q5: LeNet-like CNN for MNIST handwritten digit classification
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(6, (5, 5), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(16, (5, 5), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(120, activation="relu"),
    Dense(84, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])

model.fit(x_train, y_train_cat, epochs=10, batch_size=64,
          validation_split=0.2, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test_cat, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

predictions = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(15, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Predicted: {predicted_labels[i]}\nActual: {y_test[i]}")
```

```
    plt.axis("off")
plt.tight_layout()
plt.savefig("Q5_sample_predictions.png")
plt.show()
print("Experiment completed successfully!")
```

Question 6

Implement a neural network using different weight initialization techniques such as Xavier (Glorot) and Kaiming (He) initialization. Compare their training performance.

Algorithm

Step 1: Load MNIST, normalize and use a 20,000-sample training subset.

Step 2: Create a dictionary of initializers: Xavier -> glorot_uniform, Kaiming -> he_normal.

Step 3: For each initializer: set the same seed and build the same network (Flatten -> Dense(128, ReLU) -> Dense(10, Softmax)).

Step 4: Train both networks with identical optimizer, epochs and batch size.

Step 5: Store history (training/validation loss and accuracy) and test results.

Step 6: Plot training loss and accuracy of both initializers for comparison.

Step 7: Report which initialization trained faster / reached better accuracy.

Program (Answer06_Xavier_Kaiming_Training_Comparison.py)

```
# Q7: Compare Xavier and Kaiming initialization in a neural network
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train[:20000]
y_train = y_train[:20000]

initializers = {"Xavier": "glorot_uniform", "Kaiming": "he_normal"}
histories = {}
results = {}

for name, initializer in initializers.items():
    print(f"\nTraining using {name} initialization")
    tf.keras.utils.set_random_seed(42)
    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation="relu", kernel_initializer=initializer),
        Dense(64, activation="relu", kernel_initializer=initializer),
        Dense(10, activation="softmax")
    ])
    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    history = model.fit(x_train, y_train, epochs=8, batch_size=128,
                        validation_split=0.1, verbose=1)
    histories[name] = history.history
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results[name] = (test_loss, test_accuracy)
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")

for metric, title, filename in [
    ("accuracy", "Training Accuracy Comparison", "Q7_training_accuracy.png"),
    ("loss", "Training Loss Comparison", "Q7_training_loss.png")
]:
    plt.figure(figsize=(8, 5))
    for name in initializers:
        plt.plot(histories[name][metric], label=name)
    plt.title(title)
    plt.xlabel("Epoch")
    plt.ylabel(metric.capitalize())
    plt.legend()
```

```
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(filename)
    plt.show()

print("\nMethod\tTest accuracy\tTest loss")
for name, (loss, accuracy) in results.items():
    print(f"{name}\t{accuracy * 100:.2f}%\t\t{loss:.4f}")
print("Experiment completed successfully!")
```

Question 7

Design and implement a CNN for MNIST digit classification containing convolution, pooling, flattening and fully connected layers. Train and evaluate the model.

Algorithm

Step 1: Load MNIST, normalize and reshape to (28, 28, 1).

Step 2: Convolution: Conv2D(32, 3x3, ReLU) extracts local features.

Step 3: Pooling: MaxPooling2D(2x2) reduces spatial size.

Step 4: Repeat with Conv2D(64, 3x3, ReLU) and MaxPooling2D(2x2).

Step 5: Flatten the feature maps into a vector.

Step 6: Fully connected: Dense(128, ReLU) followed by Dense(10, Softmax).

Step 7: Compile (Adam, sparse categorical cross-entropy), display model.summary(), train and evaluate on test data.

Program (Answer07_CNN_Architecture_MNIST.py)

```
# Q8: CNN architecture with convolution, pooling, flatten and dense layers
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(x_train, y_train, epochs=10, batch_size=64,
                    validation_split=0.2, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.savefig("Q8_accuracy.png")
plt.show()
print("Experiment completed successfully!")
```

Question 8

Implement a CNN for MNIST digit recognition and visualize correctly and incorrectly classified test images.

Algorithm

Step 1: Load MNIST, normalize and reshape to (28, 28, 1).

Step 2: Build and train the CNN (two Conv+MaxPool blocks, Flatten, Dense, Softmax).

Step 3: Predict class probabilities for the test set and take argmax as predicted labels.

Step 4: Compare predicted labels with true labels to find correct and incorrect indices.

Step 5: Plot up to ten correctly classified images with true and predicted labels.

Step 6: Plot up to ten misclassified images with true and predicted labels.

Step 7: Print the test accuracy.

Program (Answer08_CNN_Correct_Incorrect_Visualization.py)

```
# Q9: CNN for MNIST digit recognition; show correct and incorrect predictions
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

tf.keras.utils.set_random_seed(42)

# 1. Load and preprocess MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

# 2. Build CNN
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

# 3. Compile and train
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(x_train, y_train, epochs=5, batch_size=64,
          validation_split=0.2, verbose=1)

# 4. Evaluate
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

# 5. Predict test images
probabilities = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(probabilities, axis=1)

correct_indices = np.where(predicted_labels == y_test)[0]
incorrect_indices = np.where(predicted_labels != y_test)[0]

print("Correctly classified images:", len(correct_indices))
print("Incorrectly classified images:", len(incorrect_indices))

# 6. Display correctly classified examples
if len(correct_indices) > 0:
```

```
    selected = correct_indices[:10]
    plt.figure(figsize=(12, 5))
    for plot_number, index in enumerate(selected, start=1):
        plt.subplot(2, 5, plot_number)
        plt.imshow(x_test[index].squeeze(), cmap="gray")
        plt.title(f"Pred: {predicted_labels[index]}\nActual: {y_test[index]}")
        plt.axis("off")
    plt.suptitle("Correctly Classified Images")
    plt.tight_layout()
    plt.savefig("Q9_correct_predictions.png")
    plt.show()

# 7. Display incorrectly classified examples
if len(incorrect_indices) > 0:
    selected = incorrect_indices[:10]
    plt.figure(figsize=(12, 5))
    for plot_number, index in enumerate(selected, start=1):
        plt.subplot(2, 5, plot_number)
        plt.imshow(x_test[index].squeeze(), cmap="gray")
        plt.title(f"Pred: {predicted_labels[index]}\nActual: {y_test[index]}")
        plt.axis("off")
    plt.suptitle("Incorrectly Classified Images")
    plt.tight_layout()
    plt.savefig("Q9_incorrect_predictions.png")
    plt.show()
else:
    print("No incorrectly classified images were found.")

print("Experiment completed successfully!")
```

Question 9

Implement RNN and LSTM models using the IMDB dataset. Compare their training and validation accuracy and explain why LSTM performs differently from Simple RNN.

Algorithm

Step 1: Load IMDb (10,000-word vocabulary); pad all reviews to length 500.

Step 2: Build SimpleRNN and LSTM models with the same embedding size and units: Embedding(32) -> SimpleRNN(32) / LSTM(32) -> Dense(1, Sigmoid).

Step 3: Compile both and train for 5 epochs with 20% validation split.

Step 4: Plot training and validation accuracy for both models.

Step 5: Explain the difference.

Program (Answer09_RNN_vs_LSTM_IMDB.py)

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, Dense

num_words, maxlen = 10000, 500
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=num_words)
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

layers = {"SimpleRNN": SimpleRNN, "LSTM": LSTM}
histories = {}

for name, layer in layers.items():
    model = Sequential([
        Embedding(num_words, 32),
        layer(32),
        Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    histories[name] = model.fit(x_train, y_train, epochs=5, batch_size=64,
                                validation_split=0.2, verbose=0)
    _, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"{name}: Test Accuracy={acc:.4f}")

plt.figure(figsize=(8, 5))
for name, h in histories.items():
    plt.plot(h.history["accuracy"], label=f"{name} Train")
    plt.plot(h.history["val_accuracy"], "--", label=f"{name} Val")
plt.xlabel("Epoch"); plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy: SimpleRNN and LSTM")
plt.legend(); plt.grid(True); plt.show()

# Explanation: SimpleRNN has a single hidden state, and gradients shrink or
# explode over long sequences (vanishing/exploding gradient), so it forgets
# early words. LSTM adds a cell state controlled by input, forget and output
# gates, which decide what to store, discard and output. This lets it retain
# long-term dependencies and usually gives more stable, higher accuracy on
# long reviews.
```

Explanation (write in exam)

SimpleRNN has a single hidden state, and gradients shrink or explode over long sequences (vanishing/exploding gradient), so it forgets early words. LSTM adds a cell state controlled by input, forget and output gates, which decide what to store, discard and output. This lets it retain long-term dependencies and usually gives more stable, higher accuracy on long reviews.

Question 10

Implement Simple RNN and GRU models for the same sequence classification problem. Compare their performance in terms of accuracy, loss and training time.

Algorithm

Step 1: Load IMDb (10,000-word vocabulary); pad all reviews to length 500.

Step 2: Build SimpleRNN(32) and GRU(32) models with the same Embedding(32).

Step 3: Compile (Adam, binary cross-entropy).

Step 4: Time training with time.perf_counter().

Step 5: Train 5 epochs, batch size 64, 20% validation.

Step 6: Evaluate test loss/accuracy; print training times and compare.

Program (Answer10_RNN_vs_GRU.py)

```
import time
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, GRU, Dense

num_words, maxlen = 10000, 500
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=num_words)
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

layers = {"SimpleRNN": SimpleRNN, "GRU": GRU}
histories, times = {}, {}

for name, layer in layers.items():
    model = Sequential([
        Embedding(num_words, 32),
        layer(32),
        Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    start = time.perf_counter()
    histories[name] = model.fit(x_train, y_train, epochs=5, batch_size=64,
                                validation_split=0.2, verbose=0)
    times[name] = time.perf_counter() - start
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    print(f"{name}: Test Loss={loss:.4f}, Test Accuracy={acc:.4f}")

for name, sec in times.items():
    print(f"{name} training time: {sec:.2f} seconds")

for name, h in histories.items():
    plt.plot(h.history["val_accuracy"], label=name)
plt.xlabel("Epoch"); plt.ylabel("Validation Accuracy")
plt.title("SimpleRNN vs GRU"); plt.legend(); plt.grid(True); plt.show()
```